

RAPPORT J03

Git/GitHub : versionnement et publication du parcours SQL/Data

Parcours	Plan d'exécution SQL/Data - 16 semaines
Semaine / Jour	Semaine 1 / J03
Date de référence	16 septembre 2026
Durée prévue	75-90 min
Timer choisi	90 min
Durée réelle	127 min
Écart	+37 min (+41,1 %)
Statut	Terminé et documenté

1. Objectifs de la séance

1. Initialiser un dépôt Git professionnel à la racine du parcours.
2. Comprendre le cycle Working Directory -> Staging Area -> Commit -> Remote.
3. Inspecter les fichiers avant staging et éviter un git add . aveugle.
4. Construire et utiliser un .gitignore.
5. Vérifier l'identité Git avant le premier commit.
6. Créer des commits logiques et contrôler l'historique.
7. Renommer la branche principale master en main.
8. Créer un dépôt GitHub vide sql-data-training.
9. Configurer le remote origin.
10. Effectuer le premier push et établir l'upstream main <-> origin/main.
11. Terminer la séance avec un dépôt local et distant synchronisé et un working tree clean.

2. Environnement et prérequis

Racine du projet :

D:\2033\SQL - TRAINING

Éléments déjà présents :

- documentation/
- Plan_Execution_SQL_Data_16_Semaines.pdf

Git :

git version 2.55.0.windows.5

Le parcours PostgreSQL des J01-J02 est déjà documenté dans documentation/semaine-01/.

3. Vérification de l'état initial

Commandes :

```
pwd
Get-ChildItem
git --version
git status
```

Résultat initial :

```
fatal: not a git repository (or any of the parent directories): .git
```

Interprétation

Ce message était attendu : le dossier SQL - TRAINING n'était pas encore un dépôt Git. Il a donc servi de preuve de l'état initial avant git init.

4. Initialisation du dépôt Git

Commande :

```
git init
git status
```

Résultats :

- création du dossier interne .git/;
- branche initiale master;
- aucun commit;
- fichiers du projet affichés comme Untracked.

Notion

git init transforme le dossier courant en dépôt Git local. Il ne publie rien sur GitHub et n'ajoute pas automatiquement les fichiers à l'historique.

5. `.gitignore` et inspection des fichiers

Le .gitignore a été préparé pour anticiper les prochaines phases SQL/Data/Python/dbt : secrets, caches, environnements virtuels, fichiers temporaires, jeux de données locaux, artefacts dbt, IDE, logs et sauvegardes.

Les scripts SQL doivent rester versionnés : ils constituent du code source.

Inspection :

```
Get-ChildItem .\documentation -Recurse -File | Select-Object FullName
git ls-files --others --exclude-standard
```

Principe retenu

Inspecter -> sélectionner -> vérifier -> commit

Cela réduit le risque de versionner accidentellement un secret, un fichier temporaire ou une donnée inutile.

6. Premier staging

Commandes :

```
git add .\gitignore
git add .\Plan_Execution_SQL_Data_16_Semaines.pdf
git add .\documentation\
git status
git status --short
git diff --cached --stat
git diff --cached --name-status
```

Résultat

Avant staging :

```
?? fichier
```

Après git add :

```
A fichier
```

A signifie Added dans la staging area.

git diff --cached --stat a permis d'évaluer le volume du futur commit et git diff --cached --name-status de contrôler précisément les fichiers stagés.

7. Warning LF -> CRLF

Pendant git add, Git a signalé notamment :

```
LF will be replaced by CRLF the next time Git touches it
```

Diagnostic documenté

- Type : Warning
- Composant : Git / fins de ligne Windows
- Cause : différence entre LF et CRLF
- Impact : non bloquant
- Décision J03 : aucune modification de configuration
- Sujet à approfondir : core.autocrlf et .gitattributes

Leçon

Un warning doit être compris et documenté avant de modifier une configuration globale.

8. Vérification de l'identité Git

Commandes :

```
git config --get user.name
git config --get user.email
git config --show-origin --get user.name
git config --show-origin --get user.email
```

Les valeurs étaient déjà configurées dans le .gitconfig global de Windows. Les informations personnelles ont été masquées dans les captures destinées à la documentation.

9. Premier commit local

Commande :

```
git commit -m "docs: initialize SQL/Data training documentation"
```

Résultat principal :

```
[master (root-commit) 68c808c]
47 files changed, 668 insertions(+)
```

Interprétation

- root-commit : premier commit du dépôt;
- 68c808c : hash abrégé du commit;
- master : branche active à ce moment;
- le commit constitue le premier snapshot local du parcours.

Historique :

```
git log --oneline --decorate
68c808c (HEAD -> master) docs: initialize SQL/Data training documentation
```

HEAD indique la position courante dans l'historique et pointe ici vers la branche active.

10. Découverte pratique : un `git add` n'est pas permanent

Les captures créées après le premier staging sont apparues comme Untracked.

Cela démontre que :

```
git add dossier/
```

place l'état courant des fichiers dans la staging area; il ne signifie pas que tous les futurs fichiers créés dans ce dossier seront automatiquement inclus.

Cycle observé :

```
Fichier créé
|
v
Untracked (??)
|
| git add
v
Staged (A)
|
| git commit
v
Committed
```

11. Deuxième commit local

Après ajout des nouvelles preuves J03 :

```
git add documentation/semaine-01/jour-03/captures/
git diff --cached --name-status
```

SQL/Data 16 semaines - S1 J03 - Rapport d'exécution

```
git commit -m "docs: add J03 Git workflow evidence"
git status
git log --oneline --decorate --graph
```

Résultat :

```
* 101191b (HEAD -> master) docs: add J03 Git workflow evidence
* 68c808c docs: initialize SQL/Data training documentation
```

À cet instant, le dépôt possédait un véritable historique à deux commits.

12. Renommage de `master` vers `main`

Commande :

```
git branch -m master main
```

Vérification :

```
git status
git log --oneline --decorate --graph
```

Résultat :

```
* 101191b (HEAD -> main) docs: add J03 Git workflow evidence
* 68c808c docs: initialize SQL/Data training documentation
```

Leçon

Le renommage de branche ne modifie pas le contenu des commits. Il déplace/renomme la référence de branche.

13. Incident : GitHub CLI indisponible

Commande :

```
gh --version
```

Erreur :

```
CommandNotFoundException
```

git remote -v ne retournait encore aucune ligne.

Diagnostic

- Git était installé et fonctionnel.
- GitHub CLI (gh) était absent ou non accessible via le PATH.
- Aucun remote n'était configuré.

Résolution

Aucune installation supplémentaire n'a été imposée pendant la séance. Le dépôt GitHub a été créé via l'interface Web.

Leçon

```
git.exe != gh.exe
```

Git est le système de versionnement; GitHub est une plateforme d'hébergement; GitHub CLI est un outil optionnel de pilotage de GitHub.

14. Création du dépôt GitHub

Dépôt créé :

```
sql-data-training
```

Description :

16-week hands-on SQL, PostgreSQL, Analytics Engineering and Data Engineering learning journey

Configuration choisie :

- visibilité : Public;
- README GitHub : désactivé;
- .gitignore GitHub : aucun;
- licence : aucune.

Pourquoi un dépôt vide ?

Le dépôt local possédait déjà son propre historique. Créer un README ou un autre fichier directement sur GitHub aurait créé un commit distant indépendant à réconcilier.

15. Configuration du remote `origin`

Commande :

```
git remote add origin https://github.com/<account>/sql-data-training.git  
git remote -v
```

Résultat conceptuel :

```
origin ... (fetch)  
origin ... (push)
```

Puis :

```
git remote show origin  
git branch -vv
```

Avant le premier push :

- origin existait;
- GitHub était encore vide;
- main n'avait pas encore d'upstream;
- le HEAD branch distant pouvait donc apparaître comme (unknown).

16. Premier push et création de l'upstream

Commande :

```
git push -u origin main
```

Résultat :

```
[new branch]    main -> main  
branch 'main' set up to track 'origin/main'.
```

Signification

git push a envoyé les commits locaux.

-u / --set-upstream a établi la relation :

```
main <-> origin/main
```

Après cela, les prochains envois peuvent normalement utiliser simplement :

```
git push
```

17. Vérification de la synchronisation

Commandes :

```
git branch -vv  
git remote show origin  
git status
```

Résultats observés :

- main suit [origin/main];
- HEAD branch: main;
- main pushes to main (up to date);
- la branche locale est à jour avec origin/main.

Des captures J03 créées après les commits restaient encore Untracked. Ce comportement a permis de distinguer :

une branche synchronisée avec son upstream

de :

la présence de nouveaux fichiers locaux non commités.

18. Vérification visuelle sur GitHub

Après actualisation du dépôt GitHub, l'interface affichait :

- branche main;
- 2 commits à ce stade;
- .gitignore;
- Plan_Execution_SQL_Data_16_Semaines.pdf;
- documentation/semaine-01.

Les captures créées après le deuxième commit n'étaient pas encore publiées.

Leçon fondamentale

git push pousse des commits. Il ne copie pas automatiquement tous les fichiers présents sur le disque local.

19. Clôture documentaire finale

Les dernières captures ont été ajoutées puis un commit documentaire final a été créé et poussé :

```
git add documentation/semaine-01/jour-03/captures/  
git status --short  
git commit -m "docs: complete J03 Git workflow documentation"  
git push  
git status
```

État final communiqué :

On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Validation finale

- dépôt local propre : oui;
- branche principale : main;
- remote origin : configuré;
- upstream : main <-> origin/main;
- commits poussés : oui;
- fichiers non suivis : aucun à la clôture.

20. Captures d'écran J03

La nomenclature retenue est séquentielle et directement réutilisable :

J03-01_git-initial-state.png
J03-02_project-root-before-git-init.png
J03-03_git-repository-initialized.png
J03-04_gitignore-and-untracked-files.png
J03-05_files-before-staging_Powershell-Command.png
J03-05_files-before-staging_Git-Command.png
J03-06_git-first-add.png
J03-06_files-after-staging_git-status.png
J03-06_files-after-staging_git-status-short.png
J03-07_staging-area-before-first-commit_git-diff-cached-stat.png
J03-07_staging-area-before-first-commit_git-diff-cached-name-status.png
J03-08_git-identity-before-first-commit.png
J03-09_first-local-commit.png
J03-10_git-log-after-first-commit.png
J03-11_second-local-commit-and-git-history.png
J03-12_main-branch-after-rename.png
J03-13_github-cli-and-remote-check.png
J03-14_github-new-repository-settings.png
J03-15_github-empty-remote-repository.png
J03-16_origin-remote-added.png
J03-17_local-branch-before-first-push.png
J03-18_first-push-to-github.png
J03-19_upstream-after-first-push.png
J03-20_github-repository-after-first-push.png

21. Commandes essentielles retenues

```
git init  
git status
```

git status --short

git add <fichier-ou-dossier>
git diff --cached --stat
git diff --cached --name-status

git config --get user.name
git config --get user.email
git config --show-origin --get user.name
git config --show-origin --get user.email

git commit -m "message"
git log --oneline --decorate --graph

git branch -m master main
git branch -vv

git remote -v
git remote add origin <URL>
git remote show origin

git push -u origin main
git push

22. Erreurs, warnings et résolutions

Événement	Diagnostic	Résolution / décision	Leçon
fatal: not a git repository	Aucun dépôt Git n'existait encore	git init	Vérifier l'état initial avant d'agir
Warning LF -> CRLF	Gestion des fins de ligne sous Windows	Pas de modification globale pendant J03	Comprendre avant de configurer
gh non reconnu	GitHub CLI absent / hors PATH	Création du dépôt via GitHub Web	Git et GitHub CLI sont distincts
Nouvelles captures Untracked après commit	Fichiers créés après le dernier staging	Nouveau staging + commit documentaire	git add capture un état, pas les fichiers futurs
HEAD branch: (unknown) avant push	Remote GitHub encore vide	Premier git push -u origin main	Une branche distante doit d'abord exister

23. Challenge J03 - validation des acquis

Être capable d'expliquer sans aide :

- la différence entre Working Directory, Staging Area et Repository;
- la signification de ?? et A dans git status --short;
- pourquoi un fichier créé après git add n'est pas automatiquement dans le commit;
- la différence entre git commit et git push;
- ce que représentent main, origin, origin/main et l'upstream;
- pourquoi le dépôt GitHub a été créé vide;
- pourquoi git push -u origin main n'est nécessaire qu'au premier établissement de l'upstream;
- pourquoi working tree clean est un bon état de clôture.

Validation pratique : réussie, puisque le workflow a été exécuté de bout en bout et vérifié localement et sur GitHub.

24. KPI de progression

KPI	Résultat J03
Durée prévue	75-90 min
Timer choisi	90 min
Durée réelle	127 min
Écart vs timer	+37 min
Dépassement relatif	+41,1 %
Dépôt Git initialisé	Oui
.gitignore utilisé	Oui
Staging inspecté avant commit	Oui
Premier commit	Oui
Historique multi-commit	Oui
Branche principale main	Oui
Remote origin	Oui
Premier push GitHub	Oui
Upstream configuré	Oui
Vérification GitHub	Oui
État final working tree clean	Oui
Incidents documentés	Oui
DDL prévu	Reporté

KPI d'amélioration

Le dépassement de 37 min par rapport au timer de 90 min est le principal point d'amélioration. Les prochaines séances doivent préserver une marge de 10-15 minutes pour la documentation et la clôture.

25. Conclusion J03

J03 a transformé le dossier local du parcours en un projet réellement versionné et publié.

Chaîne maîtrisée :

Working Directory

|

| git add

v

Staging Area

|

| git commit

v

Local Repository

|

| git push

v

GitHub / origin

État final :

main <-> origin/main

working tree clean

Le résultat est une première brique importante du portfolio : les prochains scripts SQL, Python, dbt et Data Engineering pourront être ajoutés dans un historique traçable, reproductible et publiable.

26. Prochaines étapes

À la prochaine séance :

20. commencer par git status;
21. reprendre le volet PostgreSQL/DDL reporté;
22. créer la première vraie table métier dans le schéma raw;
23. pratiquer CREATE TABLE, types, contraintes et premières opérations DML selon le plan;
24. enregistrer le script SQL dans le dépôt;
25. effectuer un commit ciblé;
26. pousser vers origin/main;
27. réserver explicitement les dernières 10-15 minutes à la documentation.

27. Potentiel de réutilisation pédagogique

Tutoriel écrit

Titre : Initialiser et publier proprement un projet Data avec Git et GitHub sous Windows

Contenu :

- audit initial;
- git init;
- .gitignore;
- staging contrôlé;
- commits;
- branche main;
- création d'un remote vide;
- origin;
- premier push;
- upstream;
- erreurs courantes.

Vidéo éducative courte

Durée : 8-12 min

Angle : « Du dossier Windows au dépôt GitHub : comprendre Git par la pratique »

Module de formation

Durée : 45-60 min

Public : débutants SQL/Data/Python, stagiaires développeurs, analystes souhaitant professionnaliser leurs projets.

Exercice réutilisable

Donner un dossier non versionné à l'apprenant et lui demander de :

- inspecter;
- créer .gitignore;
- initialiser Git;
- effectuer deux commits logiques;
- créer un dépôt distant vide;
- configurer origin;
- publier main;
- terminer avec working tree clean.

Règle de capitalisation confirmée :

Apprendre -> Exécuter -> Tester -> Capturer -> Expliquer -> Documenter -> Versionner -> Capitaliser.